
property

An attribute of a class that appears to client code to be an instance variable, but instead causes a method to be called when it is accessed.

A property allows class developers to use indirection, the way that a magician uses misdirection—the audience thinks they are seeing one thing, while behind the scenes something very different is happening. When writing a class to use property decorators, the developer writes a getter and a setter method and adds a distinct built-in decorator to each one. The first method is a getter and is preceded with the built-in `@property` decorator. The name of the method defines a name of a property to be used by client code. The second method is a setter and is preceded with the `@<name of the property>.setter` decorator. Here is a minimal sample class:

```
class Example():
    def __init__(self, startingValue):
        self._x = startingValue

    @property
    def x(self): # this is the decorated getter method
        return self._x

    @x.setter
    def x(self, value): # this is the decorated setter method
        self._x = value
```

In the `Example` class, `x` is the name of the property. After the standard `__init__()` method, the unusual thing is that we have two methods that both have the same name: the name of the property. The first method is a getter, while the second is a setter. The setter method is optional, and if it's not present, the property will be read-only.

Given the `Example` class, here is some sample client code:

```
oExample = Example(10)
print(oExample.x)
oExample.x = 20
```
